



VIT[®]
B H O P A L
www.vitbhopal.ac.in

VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

Bhopal-Indore Highway, Kothrikalan, Sehore

Madhya Pradesh - 466114

CSA4008 - APPLIED MACHINE LEARNING

REG.NO : 23BAI10917

NAME : Vijay Rajesh R

BRANCH : BTECH - AIML

SEMESTER: Fall Semester 2025-26

INDEX

Ex.NO	DATE	EXPERIMENT NAME	PAGE NO.
1.	09-07-2025	Implement Decision Tree learning	3
2.	09-07-2025	Implement Logistic Regression	6
3.		Implement classification using Multilayer perceptron	
4.		Implement classification using Multilayer perceptron	
5.		Implement Adaboost Algorithm	
6.		Implement Bagging using Random Forests	
7.		Implement K-means Clustering to Find Natural Patterns in Data	
8.		Implement Principle Component Analysis for Dimensionality Reduction	
9.		Evaluating ML algorithm with balanced and unbalanced datasets	
10.		Evaluating ML algorithm with balanced and unbalanced datasets	

EXP.NO: 01	IMPLEMENT DECISION TREE LEARNING
DATE: 09.07.25	

AIM

To build and evaluate a Decision Tree Regression model on the Insurance dataset to predict medical insurance charges.

PROCEDURE

1. Import the necessary libraries:
Use pandas, numpy for data handling; matplotlib and seaborn for visualization; and sklearn for machine learning functions.
2. Load the dataset:
Load the insurance.csv file into a pandas DataFrame using `pd.read_csv()`.
3. Preprocess the data:
 - Check for any missing values using `isnull().sum()`.
 - Convert categorical columns (like sex, smoker, and region) into numerical form using encoding methods.
4. Split the features and target:
 - Define independent features (e.g., age, sex, bmi, etc.).
 - Set the dependent variable (charges) as the target.
5. Divide the data into training and testing sets:
Use `train_test_split()` to split the data (e.g., 80% for training, 20% for testing).
6. Train the model:
Initialize and train a `DecisionTreeRegressor` on the training data.
7. Make predictions:
Use the trained model to predict charges for the test set.
8. Evaluate the model:
Calculate Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 score using sklearn metrics.

PROGRAM

```
# 1. Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
```

```

from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 2. Load the dataset
file = pd.read_csv("insurance.csv")

# 3. Check for missing values
print(file.isnull().sum())

# 4. Encode categorical variables
file['sex'] = file['sex'].map({'male': 0, 'female': 1})
file['smoker'] = file['smoker'].map({'no': 0, 'yes': 1})
file['region'] = file['region'].map({'southwest': 0,
'southeast': 1, 'northwest': 2, 'northeast': 3})

# 5. Split features and target
X = file.drop('charges', axis=1)
y = file['charges']

# 6. Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# 7. Train the Decision Tree Regressor
regressor = DecisionTreeRegressor()
regressor.fit(X_train, y_train)

# 8. Make predictions
y_pred = regressor.predict(X_test)

# 9. Evaluate the model
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("Root Mean Squared Error:", rmse)
print("R2 Score:", r2)

```

INPUT

Note : The dataset is insurance.csv which has more number of data values. So given below is an first few data from the entire dataset.

```
age,sex,bmi,children,smoker,region,charges
19,female,27.9,0,yes,southwest,16884.924
18,male,33.77,1,no,southeast,1725.5523
28,male,33,3,no,southeast,4449.462
33,male,22.705,0,no,northwest,21984.47061
```

OUTPUT

```
MSE : 47329009.57964182
RMSE : 6879.608243180844
R^2 : 0.7011766883576342
```

RESULT

The Decision Tree Regressor model yielded a Mean Squared Error (**MSE**) of **47,329,009.58**, a Root Mean Squared Error (**RMSE**) of **6,879.61**, and an **R²** score of **0.70**. This indicates that the model explains around **70%** of the variance in the insurance charges, providing a moderately accurate prediction, though there is room for improvement through hyperparameter tuning or more advanced models.

EXP.NO: 02	IMPLEMENT LOGISTIC REGRESSION
DATE: 09.07.25	

AIM

To implement Logistic Regression using Python for binary classification and evaluate its performance on a dataset.

PROCEDURE

1. Import Required Libraries

Import necessary Python libraries such as pandas, numpy, matplotlib, and scikit-learn.

2. Load the Dataset

Read the dataset using `pandas.read_csv()` and display the top rows using `.head()`.

3. Check for Missing Values

Use `.isnull().sum()` to verify if the dataset has any missing data.

4. Data Preprocessing

Convert categorical columns into numerical format using `get_dummies()` or `LabelEncoder` if required.

5. Split Features and Target

Define X (features) and y (target variable).

6. Split Data into Train and Test Sets

Use `train_test_split()` to divide the dataset (e.g., 80% training and 20% testing).

7. Import and Train the Logistic Regression Model

Create an instance of `LogisticRegression` and fit it to the training data using `.fit()`.

8. Make Predictions

Predict the target on the test set using `.predict()`.

9. Evaluate the Model

Evaluate model performance using metrics such as `accuracy_score`, `confusion_matrix`, and `classification_report`.

10. Display Results

Print all evaluation metrics and visualize confusion matrix (optional).

PROGRAM

```
#1-----
import pandas as pd
import numpy as np

file = pd.read_csv("insurance.csv")
file.head(3)

#2-----
file.isnull().sum()
file.head(3)

#3-----
n_file = pd.get_dummies(file, drop_first=True)
n_file.head(3)

#4-----
x = n_file.drop('charges', axis=1)
y = n_file['charges']

#5-----
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.2)

#6-----
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(x_train, y_train)

#7-----
y_pred = model.predict(x_test)

#8-----
from sklearn.metrics import mean_squared_error, r2_score

mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MSE : ", mse)
print("RMSE : ", rmse)
print("R2 Score : ", r2)
```

INPUT

```
age,sex,bmi,children,smoker,region,charges
52,female,44.7,3,no,southwest,11411.685
50,male,30.97,3,no,northwest,10600.5483
18,female,31.92,0,no,northeast,2205.9808
18,female,36.85,0,no,southeast,1629.8335
21,female,25.8,0,no,southwest,2007.945
61,female,29.07,0,yes,northwest,29141.3603
```

OUTPUT

```
MSE : 30218435.943220887
RMSE : 5497.1297913748485
R^2 : 0.8097890765450464
```

RESULT

The Linear Regression Model achieved a Mean Squared Error (**MSE**) of approximately **30,218,435.94**, a Root Mean Squared Error (**RMSE**) of around **5497.13**, and an **R²** score of **0.81**. This indicates that the model explains **81%** of the variance in the target variable, showing good predictive performance and a reasonably accurate fit for the dataset used.